Nama : Afriza Fastaqimummalka

Nim : 2341053

Prodi : Teknik Informatika

Matkul : Sistem Terbagi

---

1) **Exercise 1.1 — Perbedaan Utama Sistem Paralel dan Sistem Terdistribusi**

   Perbedaannya berfokus pada beberapa klasifikasi utamanya :

   a. **Sistem Paralel**

      - Multiprocessor System — Banyak prosesor dengan akses langsung ke shared memory yang membentuk common address space. Prosesor-prosesor ini biasanya tidak memiliki common clock tersendiri karena berbagi memori sudah cukup untuk sinkronisasi.
      - Multicomputer System — Setiap komputer punya memori sendiri, namun terhubung melalui jaringan interkoneksi berkecepatan tinggi dan beroperasi di bawah satu sistem operasi tunggal.
      - Array Processor — Banyak prosesor identik yang bekerja secara sinkron di bawah kendali satu unit kontrol tunggal.

   b. **Sistem Terdistribusi**

      - No common physical clock — Tidak ada clock fisik bersama. Ini adalah asumsi penting karena memunculkan elemen 'distribusi' dan menyebabkan asinkronisasi inherent antar prosesor.
      - No shared memory — Fitur kunci yang mengharuskan komunikasi via message-passing. Ketiadaan shared memory ini secara implisit juga berarti tidak ada common physical clock.
      - Geographical separation — Prosesor tersebar secara geografis. Namun buku menegaskan bahwa prosesor tidak harus berada di WAN; konfigurasi NOW/COW (Network/Cluster of Workstations) di LAN pun sudah dianggap sebagai distributed system kecil.
      - Autonomy and heterogeneity — Prosesor bersifat loosely coupled, dapat memiliki kecepatan berbeda, dan masing-masing dapat menjalankan sistem operasi yang berbeda.

2) **Exercise 1.2 — Aplikasi Terdistribusi dan Faktor Motivasi**

a. **Secara eksplisit ada tujuh faktor motivasi penggunaan sistem terdistribusi :**

1. Inherently Distributed Computations — Banyak komputasi yang secara alami bersifat terdistribusi, seperti transfer uang perbankan atau pencapaian konsensus antar pihak yang berjauhan secara geografis.
2. Resource Sharing — Sumber daya seperti periferal, dataset di basis data, dan library khusus tidak bisa direplikasi sepenuhnya di semua lokasi karena tidak praktis dan tidak efisien secara biaya.
3. Access to Geographically Remote Data and Resources — Data tertentu tidak bisa direplikasi di setiap lokasi karena terlalu besar atau terlalu sensitif. Contoh: data payroll multinasional disimpan di server pusat dan diakses oleh cabang-cabang.
4. Enhanced Reliability — Reliability mencakup tiga aspek: (a) availability — sumber daya dapat diakses kapan saja; (b) integrity — nilai/state sumber daya benar meski diakses bersamaan; (c) fault-tolerance — kemampuan pulih dari kegagalan.
5. Increased Performance/Cost Ratio — Dengan resource sharing, konfigurasi distributed sistem memberikan rasio performa/biaya lebih baik dibanding menggunakan mesin paralel khusus yang mahal.
6. Scalability — Karena prosesor umumnya terhubung melalui WAN, penambahan prosesor tidak langsung menjadi bottleneck pada jaringan komunikasi.
7. Modularity and Incremental Expandability — Prosesor heterogen dapat ditambahkan tanpa memengaruhi performa, selama menjalankan algoritma middleware yang sama

b. **Aplikasi dan Pemetaan Faktor Motivasi**

No	Faktor Motivasi	Aplikasi Ilmiah	Aplikasi Komersial
1	Inherently Distributed Computations	Transfer uang perbankan lintas cabang, konsensus multi-pihak	Sistem banking SWIFT, transfer antar bank
2	Resource Sharing	Basis data genomik bersama antar laboratorium riset	E-commerce (katalog & inventaris bersama)
3	Access to Geographically Remote Data	Akses supercomputer di lokasi berbeda untuk simulasi	Payroll multinational, akses data cabang ke server pusat
4	Enhanced Reliability	Replikasi data eksperimen kritis, fault-tolerance	Sistem perbankan/ATM harus available 24/7
5	Increased Performance/Cost Ratio	Komputasi grid (SETI@home, LHC) memanfaatkan idle CPU	Server farm lebih murah dari superkomputer tunggal
6	Scalability	Volume data ilmiah terus bertambah, mudah tambah node	Jutaan user e-commerce, skala sesuai demand
7	Modularity & Incremental Expandability	Tambah node baru tanpa hentikan eksperimen	Tambah server tanpa gangguan layanan

3) **Exercise 1.6 — Bukti Non-Blocking pada Jaringan Benes**

a. **Konstruksi Jaringan**

Butterfly network dengan $n = 2^k$ input memiliki $\log_2 n$ stage. Setiap stage terdiri dari n/2 switch 2×2. Masalah yang diamati di Figure 1.4 buku: dua jalur berbeda bisa berbagi sebuah edge, sehingga tidak bisa digunakan bersamaan.

Contoh konkret dari soal (n = 8, k = 3):

- Jalur input 000 → output 111
- Jalur input 101 → output 110

Kedua jalur ini melewati edge yang sama di tengah jaringan, sehingga tidak bisa digunakan simultan. Inilah yang disebut blocking network**.**Lalu Jaringan Benes dibentuk dengan menggabungkan Butterfly network dan mirror image-nya:

Butterfly ($\log_2 n$ stage) + Mirror Butterfly ($\log_2 n$ stage) = $2 \cdot \log_2 n$ stage

Setiap switch adalah unit 2×2 yang dapat dikonfigurasi straight atau cross. Apa teorema dasarnya ?

*Jaringan Benes dengan $2 \cdot \log_2 n$ stage adalah **non-blocking** — untuk setiap permutasi π pada {0, ..., n√1}, selalu ada routing simultan tanpa konflik.*

b. **Bukti (Induksi pada k, $n = 2^k$)**

$$k = 1,\ n = 2$$

$$\text{Stage} = 2\log_2(n) = 2\log_2(2) = 2(1) = 2$$

$$\text{Switch} = 1 \text{ buah } (2 \times 2)$$

Dua permutasi yang mungkin (identitas dan swap) dilayani dengan konfigurasi straight dan cross. Basis terbukti.

- **Hipotesis Induksi (HI)**
  Asumsikan Benes network berukuran n/2 adalah non-blocking.
- **Langkah Induksi: $n = 2^k$**
  - Benes network berukuran n memiliki struktur rekursif:

$$[\text{Kolom-1 } (n/2\ switch)]$$

$$+[\text{Benes}(n/2)\ atas]$$

$$+[\text{Benes}(n/2)\ bawah]$$

$$+[\text{Kolom terakhir } (n/2\ switch)]$$

- Maka Jumlah stage:

$$1 + 2 \cdot \log_2\left(\frac{n}{2}\right) + 1$$

$$= 1 + 2(\log_2 n - 1) + 1$$

$$= 2\log_2 n$$

Diberikan permutasi π sembarang. Routing dilakukan sebagai berikut:

1. Pilih input 0, tetapkan ke subnetwork **atas**. Maka partner-nya di switch yang sama (input 1) harus ke subnetwork **bawah**.
2. Lihat output π(0): harus tiba dari subnetwork atas. Partner-nya di kolom terakhir harus tiba dari subnetwork bawah. Input yang menuju partner tersebut ditetapkan ke bawah, dan partner-nya ke atas.
3. Ulangi propagasi ini hingga semua input tertugaskan.

Proses ini tidak pernah menghasilkan kontradiksi karena:

- π adalah **bijeksi** → tidak ada dua input menuju output yang sama.
- Graf constraint yang terbentuk adalah **bipartite reguler berderajat 2** → selalu dapat didekomposisi menjadi dua perfect matching (atas dan bawah).

Maka didapatlan hasilnya: n/2 input ke subnetwork atas dan n/2 ke bawah, masing-masing membentuk sub-permutasi berukuran n/2. Berdasarkan HI, kedua subnetwork mampu melayaninya secara non-blocking.